SentinelIQ 08 — Data Model & API Specification

Oct 13, 2026 · @maxi

The data model has eight tables, and the versioned REST API maps its routes to requirements FR1–FR8.

Tables

Table	Key columns	Purpose
users	id, email, password_hash, role, created_at	Accounts and roles (FR1)
events	id, dedup_key, source, occurred_at, ingested_at, event_type, features, raw, label	Normalized events from every source (FR2, FR3)
jobs	id, event_id, kind, status, attempts, last_error, run_after	Work queue for scoring and triage
model_versions	id, name, version, trained_at, metrics, artifact_path	Registry of trained models (FR4)
scores	id, event_id, model_version_id, risk, scored_at	One score per event per model version (FR4)
explanations	id, event_id, summary, technique_ids, retrieved, prompt_version, llm_name, created_at	Cited explanations with their sources (FR5)
knowledge_chunks	id, technique_id, title, text, embedding, attack_release	ATT&CK content used for retrieval (FR5)
audit_log	id, event_id, user_id, action, note, created_at	Immutable analyst decisions (FR7)

The `label` column on `events` is read only by evaluation code, never by scoring or triage.

Constraints and indexes

- `events.dedup_key` is unique, which makes ingestion idempotent (FR2).
- `users.email` is unique, and `users.role` is limited by a check constraint to analyst, lead or admin (FR1).
- `scores` is unique on event and model version, so re-scoring with a new model adds a row instead of overwriting history.
- Foreign keys link `scores`, `explanations`, `jobs` and `audit_log` to `events`.
- The application's database account may only insert into and read from `audit_log`; it has no update or delete permission (FR7).
- Indexes: `events(occurred_at)`, `scores(risk)` for the alert queue, `jobs(status, run_after)` for claiming work, and a vector index on `knowledge_chunks(embedding)`.

API conventions

- **Versioning:** every route lives under `/api/v1`; a breaking change gets `/api/v2`.
- **Authentication:** `Authorization: Bearer <JWT>`, with short-lived access tokens issued by `POST /api/v1/auth/login`.
- **Authorization:** the role is checked on every route, and a route with no explicit rule is denied.
- **Errors:** a JSON body with `code`, `message` and `request_id`, and the right HTTP status (401, 403, 404, 409, 422).
- **Pagination:** the alert queue uses `limit` and `cursor` parameters instead of page numbers, so results stay stable while new events arrive.
- **Idempotency:** sending the same event twice returns the existing event with `duplicate: true` instead of an error.
- **Tracing:** every response carries an `X-Request-ID` header that also appears in the logs.
- **Documentation:** FastAPI generates the OpenAPI document; the interactive docs page is open locally and restricted in production.

Endpoints

All paths are under `/api/v1`.

Method	Path	Who may call it	Requirement
POST	/auth/login	Anyone	FR1
POST	/users	Admin	FR1
GET	/users	Admin	FR1
POST	/events	Authenticated service account	FR2, FR3
POST	/events/batch	Authenticated service account	FR2, FR3
GET	/alerts	Analyst, lead, admin	FR6
GET	/events/{id}	Analyst, lead, admin	FR4, FR5, FR6
POST	/events/{id}/decisions	Analyst, lead	FR7
GET	/events/{id}/decisions	Analyst, lead, admin	FR7
GET	/metrics/summary	Lead, admin	FR8
GET	/health/live, /health/ready	Anyone	Reliability (Document 03)

Event ingestion contract

Example request to `POST /api/v1/events` for an authentication event. The server returns `201` with `event_id` and `duplicate: false`, or `200` with `duplicate: true` for a repeat.

```
{
  "source": "auth_log",
  "occurred_at": "2026-10-13T08:41:07Z",
  "event_type": "login_failed",
  "username": "a.sharma",
  "src_ip": "203.0.113.45",
  "raw": { "message": "Failed password for a.sharma" }
}
```

Validation rules (proposed limits shown in brackets)

- `source` must be `network_flow` or `auth_log`; unknown top-level fields are rejected.
- `occurred_at` must be ISO 8601 in UTC and not in the future (a small clock tolerance, for example 5 minutes).
- A `network_flow` event must carry `features` with the expected numeric keys; an `auth_log` event must carry `event_type` from a known list and a `username`. The worker computes authentication features later.
- Size limits protect the API: one event up to 64 KB, one batch up to 500 events.
- The deduplication key is built by the server from the source, the time and a hash of the raw content; clients cannot choose it.

Migrations and retention

- **Migrations:** every schema change is a versioned migration file (for example with Alembic), reviewed like code.
- **Safe order:** migrations run before the new release starts and must work with the previous release, so changes add first and remove later.
- **Seed data:** a documented script loads the sample events and the ATT&CK knowledge base for local runs and the demo.
- **Retention:** for this project all events and scores are kept. The application never deletes from `audit_log`. A real deployment would set retention periods by policy.
- **Personal data:** usernames and addresses in the data are fictional or from a public dataset.
- **Backups:** out of scope for v1, as noted in Document 10.